What is QA?

Quality Assurance (QA) is the activity of ensuring quality through the development and monitoring of software development processes. The goal of QA is to prevent errors from the very beginning, when the product is still in the design phase, rather than waiting until completion to discover mistakes.

QA acts as a quality management system, helping development teams work methodically, with standards, and in a verifiable manner. QA activities include:

- Establish development processes, coding guidelines, and review standards.
- Develop a set of quality monitoring indicators such as*defect density*, *reopen rate*, *defect leakage*.
- Verify compliance with procedures by the Dev, QC, and PM teams.
- Assess risks and propose improvements when deviations are detected.

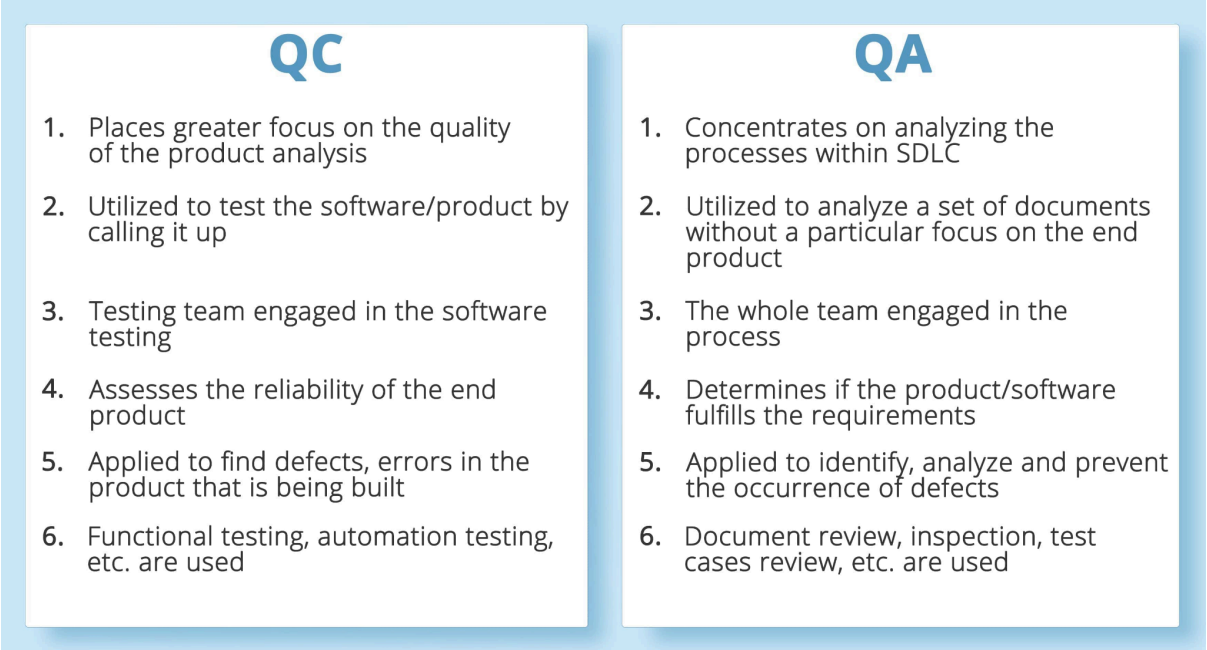
In modern models (Agile or DevOps), QA doesn't just work at the end of the project but participates from the beginning of the project lifecycle. QA works with BA and PM to clarify requirements, designs test plans with Dev, and analyzes test results with QC after each sprint. This approach ensures that the product not only meets requirements but also achieves high performance and stability.

What is QC?

Quality Control (QC) is the activity of inspecting, evaluating, and verifying the quality of a product at the output. If QA builds a prevention system, QC is the team that "measures in practice" whether the product meets that standard. QC is usually performed after a runnable version of the software has been built or before the official release. The main tasks of QC include:

- Plan and execute various types of tests (manual, automation, performance, regression, etc.).
- Record the errors and categorize them by severity (critical, major, minor, cosmetic).
- Monitor the bug fixing process and retest.
- Assess the overall level of product readiness.

Quality control (QC) not only detects bugs but also evaluates usability, performance, and security. A professional QC team not only "finds errors" but also provides quantitative evidence to help QA and PM improve processes and minimize recurring errors.



QA process in software development

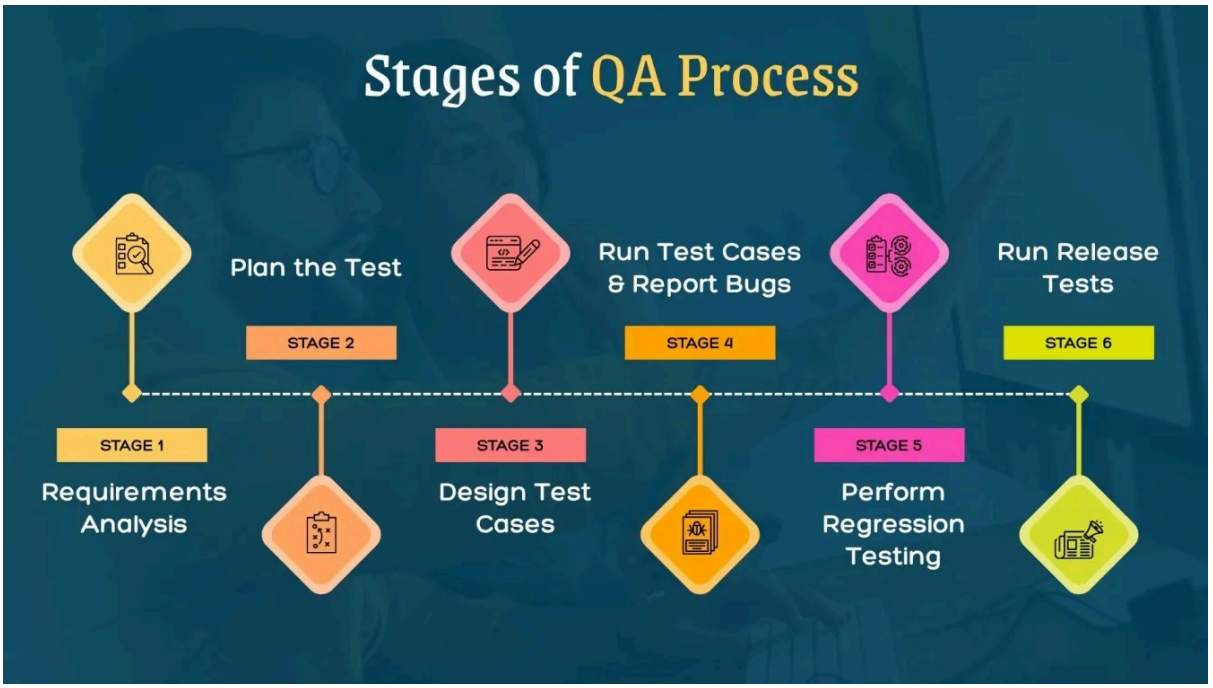
Analyze requirements and define quality standards.

This is a foundational phase that largely determines the effectiveness of the entire subsequent QA process. QA professionals work with the BA and PO teams to analyze requirements, determine clarity, completeness, and testability. A requirement is considered to meet QA standards when it can:

- Measurable using specific indicators.
- There are clear pass/fail criteria.
- It does not conflict with other requirements.

For example, instead of "fast-responding application," QA needs to clarify it to "API responds in under 500ms under 1,000 concurrent users." Only when the requirements are quantified will the team have a basis for testing and evaluation.

At this stage, QA also helps predict technical risks, such as resource shortages, reliance on third parties, or areas prone to errors. Early identification allows the Project Manager to plan for efficient resource allocation and reduce costs.



Establish a test plan and design test scenarios.

Once the requirements are finalized, QA is responsible for developing a Test Plan – a comprehensive document describing how the testing will be performed. A Test Plan typically includes:

- Test scope: which modules and functionalities are tested, and which are not.
- Testing types: manual, automation, functional, non-functional.
- Testing tools and environment.
- Implementation schedule and responsible persons.
- Acceptance Criteria.

Next, QA designs detailed test cases. Each test case simulates a specific scenario, with input data, user actions, and expected results. This allows QC to accurately reproduce errors during testing, resulting in clear and consistent reporting and bug fixing.

In large organizations, QA also develops test scenarios (scenarios that combine multiple test cases) and uses traceability matrices to ensure that all requirements are thoroughly tested without omission.

Monitor the development process and ensure compliance.

Throughout the coding phase, QA monitors the process to ensure standards are being met. Typical activities include:

- **Code Review:**Detect logic errors or coding rule violations early.
- **Static Code Analysis:**Use tools like SonarQube to assess the quality of the source code.
- **CI/CD Integration:**Ensure that each commit is automatically tested before merging.
- **Defect Trend Tracking:**Monitor bug rates by module to identify high-risk areas.

If QA detects an increasing error trend, they can suggest process modifications, add checklists, or request a more thorough review at the design stage. This is how QA not only “controls” but also leads internal process improvement.

Continuous process evaluation and improvement

After each development iteration, QA compiles test results and bug reports to assess overall quality. From there, QA performs Root Cause Analysis (RCA) to identify the root cause of errors, for example, missing information in requirements, design flaws, or programming that doesn't adhere to guidelines.

The RCA results are used to update the QA process, helping to reduce errors in subsequent projects. This is a crucial part of enabling the organization to learn from its own quality data, moving towards standardization according to models such as CMMI or ISO 9001.

Quality Control (QC) process in software development

Unit Testing

QC and Dev verify the correctness of each small function. Unit testing ensures that each module operates independently as designed. This phase is often automated, running in a CI/CD pipeline to detect errors immediately after the code is pushed to the repository.

A good set of unit tests must cover all boundary cases and exception handling. This significantly reduces the amount of manual testing later on and lays the foundation for consistent code quality.

Integration Testing

After the modules are coupled, Integration Test helps detect errors in the interaction between the parts. QC checks whether the data is transmitted in the correct format, the processing logic is consistent, and the system operates stably when dependent modules fail.

This is a crucial stage, as most serious errors typically occur at the "interaction points" between systems or services. QA and QC often collaborate to determine the appropriate test scope, avoiding under-testing or overlapping.

System testing

System testing verifies that the entire application functions as a unified whole. Quality control (QC) simulates real-world user behavior, performing actions such as registration, login, payment, or data retrieval. The goal is to confirm that the system meets technical standards and operates smoothly in the deployment environment.
In addition to functional testing, QC also performs:

- **Performance Test:**Measure speed, load capacity, and stability.
- **Security Test:**Check for security vulnerabilities according to OWASP.
- **Compatibility Test:**Ensure the application runs smoothly across multiple devices and browsers.

System testing is often seen as the "big picture" that helps engineering leaders assess the readiness of a product.

User Acceptance Testing (UAT)

UAT is the final stage before software is officially deployed. QC assists the Product Owner or customer in verifying whether the product meets real-world needs. Unlike technical testing, UAT focuses on the business perspective: whether the process is logical, the interface is user-friendly, and the results meet the intended purpose.

The UAT results are typically compiled by QA into a "go-live readiness" report, showing the level of completion and any remaining risks. When the UAT criteria are met, the product can be deployed with the highest level of confidence.

Automation Testing

Automation testing is applied to speed up and reduce human error in iterative testing. QC builds test scripts for regression testing, smoke testing, and performance testing using tools such as Selenium, Cypress, Appium, or Playwright.

By integrating automation into CI/CD, each time new code is merged, the system automatically tests and detects errors within minutes. This allows the team to release multiple times a day while maintaining consistent quality.

The collaboration between QA and QC in quality management.

QA and QC do not operate independently. When QC detects an error, the information is passed back to QA for root cause analysis and updating of preventative procedures. Conversely, QA assists QC in designing test strategies and evaluation standards.

This close collaboration creates a continuous quality improvement cycle – a hallmark of mature software companies.

For example, if QC notices an increase in UI bug rates, QA can add a checklist for UI/UX reviews or request training for developers on responsive design. This not only fixes bugs but also eliminates their root causes.

Benefits of a professional QA/QC process

A well-structured QA/QC process delivers significant value to both the developing business and its customers.

- **Reduce error correction costs:**Early detection of faults can save 60–80% on maintenance costs.
- **Speed up the release:**Automation and CI/CD help shorten test time without compromising quality.
- **Enhance the user experience:**The product has fewer defects, higher performance, and greater stability.
- **Building brand reputation:**Consistent quality builds customer trust and fosters long-term cooperation.
- **Easy to scale:**A clear process helps to speed up onboarding and maintain consistent quality across multiple projects.